龙芯 2K300/301 系列板卡使用手册-UDP 网络通信

龙芯 2K300/301 开源库使用说明

UDP 网络通信篇

一、UDP 模块简介

UDP（User Datagram Protocol，用户数据报协议）是一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。龙芯 2K300/301 开源库提供了 UDP 客户端封装，适配开发板的网络通信需求，简化开发流程。

1.1 核心功能

- 发送/接收原始字节数据
- 发送/接收字符串
- 发送 OpenCV 图像（JPEG 压缩）
- 线程安全操作
- 自动资源管理

1.2 适用场景

- 实时数据传输（传感器数据、编码器数据等）
- 图像流传输（摄像头画面）
- 简单的网络通信场景

重要说明：UDP 协议不保证数据传输的可靠性，可能出现丢包、乱序情况，重要数据建议使用 TCP 协议或添加自定义确认机制。

二、包含头文件

在使用 UDP 功能之前，需在代码中包含对应的头文件，该头文件定义了 UDP 客户端类 lq_udp_client，提供所有核心接口。

```
c++
#include "lq_udp_client.hpp"
```

三、lq_udp_client 类接口说明

3.1 构造函数

【无参构造函数】

```
c++
lq_udp_client()
```

说明：创建未初始化的 UDP 客户端对象，需后续调用 udp_client_init() 完成初始化。

【有参构造函数】

```
c++
lq_udp_client(const std::string _ip, uint16_t _port)
```

参数说明：

- _ip - 目标服务器 IP 地址（如 "192.168.1.100"）
- _port - 目标服务器端口号（如 8080）

说明：创建对象的同时完成初始化，无需额外调用初始化函数。

【构造函数示例】

```
c++
// 方式 1：无参构造 + 初始化
lq_udp_client client;
client.udp_client_init("192.168.1.100", 8080);

// 方式 2：有参构造（推荐，简洁高效）
lq_udp_client client("192.168.1.100", 8080);
```

3.2 初始化函数

```c++
void udp_client_init(const std::string _ip, uint16_t _port)
```

参数说明:

- _ip - 目标服务器 IP 地址
- _port - 目标服务器端口号

功能说明:

- 创建 UDP 套接字
- 配置目标地址和端口
- 设置端口复用选项，避免端口占用冲突

示例:

```c++
lq_udp_client udp;
udp.udp_client_init("192.168.43.98", 8080);
```

3.3 数据发送函数

【发送原始数据】

```c++
ssize_t udp_send(const void *_buf, size_t _len)
```

参数说明:

- _buf - 发送数据缓冲区指针
- _len - 发送数据长度（单位：字节）

返回值:

- 成功：返回发送的字节数
- 失败：返回负数值

示例:

```c++
uint8_t data[100] = {1, 2, 3, 4, 5};
ssize_t sent = udp.udp_send(data, sizeof(data));
```

【发送字符串】

```c++
ssize_t udp_send_string(const std::string &_str)
```

参数说明:

- _str - 要发送的字符串

返回值: 与 udp_send 一致

示例:

```c++
udp.udp_send_string("Hello World!");
udp.udp_send_string("ch1:123.45,ch2:678.90"); // 传感器数据格式示例
```

【发送图像（需 OpenCV 支持）】

```c++
ssize_t udp_send_image(const cv::Mat &_img, int _quality = 80)
```

参数说明:

- _img - OpenCV 图像（cv::Mat 格式）
- _quality - JPEG 编码质量（1-100），默认值为 80，值越高画质越好、数据量越大

返回值: 与 udp_send 一致

协议格式: 4 字节长度（小端）+ JPEG 数据

示例:

```c++
cv::Mat frame = cv::imread("test.jpg");
udp.udp_send_image(frame, 50);  // 以质量 50 发送图像
```

3.4 数据接收函数

```c++
ssize_t udp_recv(void *_buf, size_t _len)
```

参数说明:

- _buf - 接收数据缓冲区指针
- _len - 缓冲区最大长度（需确保足够容纳接收数据）

返回值：与 udp_send 一致

说明：

- 阻塞式接收，若无数据则一直等待
- 接收完成后需手动添加字符串结束符（针对字符串数据）

示例：

```
c++
char buffer[1024];
ssize_t len = udp.udp_recv(buffer, sizeof(buffer));
if (len > 0) {
    buffer[len] = '\0';  // 添加字符串结束符
    printf("Received: %s\n", buffer);
}
```

3.5 辅助函数

【获取套接字描述符】

```
c++
int get_udp_socket_fd() const
```

返回值：UDP 套接字文件描述符，可用于自定义套接字操作。

【关闭连接】

```
c++
void udp_close()
```

功能：主动关闭 UDP 套接字，释放资源。

说明：通常不需要手动调用，对象析构时会自动关闭套接字。

四、基本使用示例

4.1 发送字符串

【示例 1】发送简单字符串

```c++
#include "lq_udp_client.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 创建 UDP 客户端并初始化
    lq_udp_client udp;
    udp.udp_client_init("192.168.43.98", 8080);

    // 发送字符串
    udp.udp_send_string("Hello from Loongson!");
    printf("Message sent!\n");

    return 0;
}
```

【示例 2】循环发送数据

```c++
#include "lq_udp_client.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    // 有参构造初始化 UDP 客户端
    lq_udp_client udp("192.168.43.98", 8080);
    int count = 0;
    char msg[100];

    while(1)
    {
        // 格式化字符串（计数信息）
        snprintf(msg, sizeof(msg), "Count: %d", count);
        // 发送数据
        udp.udp_send_string(msg);
        printf("Sent: %s\n", msg);
```

```
        count++;
        sleep(1);  // 1 秒发送一次
    }

    return 0;
}
```

【示例 3】发送传感器数据

```c++
#include "lq_udp_client.hpp"
#include <stdio.h>
#include <unistd.h>

int main()
{
    lq_udp_client udp("192.168.1.100", 8080);

    while(1)
    {
        // 模拟读取传感器数据（温度、湿度、气压）
        float temperature = 25.5;
        float humidity = 60.0;
        float pressure = 101.3;

        // 格式化数据为字符串
        char data[128];
        snprintf(data, sizeof(data),
                 "temp:%.2f,hum:%.2f,pres:%.2f",
                 temperature, humidity, pressure);

        // 发送数据并判断是否成功
        ssize_t sent = udp.udp_send_string(data);
        if (sent < 0) {
            printf("Send failed!\n");
        } else {
            printf("Sent: %s (%zd bytes)\n", data, sent);
        }

        usleep(100000);  // 100ms 发送一次
    }

    return 0;
```

```
}
```

4.2 发送原始数据

【示例 4】发送结构体数据

```c++
#include "lq_udp_client.hpp"
#include <stdio.h>

// 定义传感器数据结构体
struct SensorData {
    uint32_t timestamp;       // 时间戳
    float acc_x, acc_y, acc_z;// 加速度数据
    float gyro_x, gyro_y, gyro_z;// 陀螺仪数据
};

int main()
{
    lq_udp_client udp("192.168.1.100", 8080);
    SensorData data;

    // 初始化结构体数据
    data.timestamp = 0;
    data.acc_x = 0.1;
    data.acc_y = 0.2;
    data.acc_z = 9.8;
    data.gyro_x = 0.01;
    data.gyro_y = 0.02;
    data.gyro_z = 0.03;

    // 发送结构体（直接发送内存数据）
    ssize_t sent = udp.udp_send(&data, sizeof(data));
    printf("Sent %zd bytes\n", sent);

    return 0;
}
```

【示例 5】发送数组数据

```c++
```

```
#include "lq_udp_client.hpp"
#include <stdio.h>

int main()
{
    lq_udp_client udp("192.168.1.100", 8080);

    // 发送整数数组
    int values[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    udp.udp_send(values, sizeof(values));

    // 发送字节数组
    uint8_t bytes[256];
    for (int i = 0; i < 256; i++) {
        bytes[i] = i;
    }
    udp.udp_send(bytes, sizeof(bytes));

    return 0;
}
```

4.3 接收数据

【示例 6】接收服务器响应

```
c++
#include "lq_udp_client.hpp"
#include <stdio.h>
#include <string.h>

int main()
{
    lq_udp_client udp("192.168.1.100", 8080);

    // 发送请求数据
    udp.udp_send_string("GET_DATA");

    // 接收服务器响应
    char buffer[1024];
    ssize_t len = udp.udp_recv(buffer, sizeof(buffer) - 1);
    if (len > 0) {
        buffer[len] = '\0';  // 添加字符串结束符
```

```
        printf("Received: %s\n", buffer);
    }

    return 0;
}
```

五、发送图像示例（需 OpenCV 支持）

5.1 发送静态图片

```
c++
#include "lq_udp_client.hpp"
#include <opencv2/opencv.hpp>
#include <stdio.h>

int main()
{
    // 初始化 UDP 客户端
    lq_udp_client udp("192.168.43.98", 8080);

    // 读取本地图片
    cv::Mat image = cv::imread("test.jpg");
    if (image.empty()) {
        printf("Failed to load image!\n");
        return -1;
    }

    // 缩小图片（可选，减少传输数据量，提升传输速度）
    cv::Mat resized;
    cv::resize(image, resized, cv::Size(320, 240));

    // 发送图片，JPEG 质量设为 50
    ssize_t sent = udp.udp_send_image(resized, 50);
    printf("Sent %zd bytes\n", sent);

    return 0;
}
```

5.2 发送摄像头视频流

```c++
#include "lq_udp_client.hpp"
#include <opencv2/opencv.hpp>
#include <stdio.h>
#include <unistd.h>

// 配置参数（可根据实际需求修改）
const std::string TARGET_IP   = "192.168.43.98";
const uint16_t    TARGET_PORT = 8080;
const int         CAM_WIDTH   = 160;
const int         CAM_HEIGHT  = 120;
const int         JPEG_QUALITY = 30;

int main()
{
    printf("========================================\n");
    printf("  UDP Camera Stream\n");
    printf("========================================\n");
    printf("Target: %s:%d\n", TARGET_IP.c_str(), TARGET_PORT);
    printf("Resolution: %dx%d\n", CAM_WIDTH, CAM_HEIGHT);
    printf("========================================\n");

    // 初始化 UDP 客户端
    lq_udp_client udp;
    udp.udp_client_init(TARGET_IP, TARGET_PORT);

    // 初始化摄像头（打开默认摄像头）
    cv::VideoCapture cap(0);
    cap.set(cv::CAP_PROP_FRAME_WIDTH, CAM_WIDTH);
    cap.set(cv::CAP_PROP_FRAME_HEIGHT, CAM_HEIGHT);

    if (!cap.isOpened()) {
        printf("Failed to open camera!\n");
        return -1;
    }
    printf("Camera opened. Press Ctrl+C to stop.\n");

    cv::Mat frame;
    int frame_count = 0;

    while (true) {
        // 获取摄像头画面
        cap >> frame;
        if (frame.empty()) continue;
```

```
        // 发送画面
        ssize_t sent = udp.udp_send_image(frame, JPEG_QUALITY);
        if (sent > 0) {
            frame_count++;
        }

        // 每秒打印一次传输状态
        if (frame_count % 30 == 0) {
            printf("Sent frame %d, size: %zd bytes\n",
frame_count, sent);
        }

        usleep(30000);  // 30ms 间隔，约 30fps
    }

    return 0;
}
```

5.3 发送处理后的图像

```
c++
#include "lq_udp_client.hpp"
#include <opencv2/opencv.hpp>
#include <stdio.h>

int main()
{
    lq_udp_client udp("192.168.43.98", 8080);
    cv::VideoCapture cap(0);

    // 设置摄像头分辨率
    cap.set(cv::CAP_PROP_FRAME_WIDTH, 320);
    cap.set(cv::CAP_PROP_FRAME_HEIGHT, 240);

    cv::Mat frame;
    while (true) {
        cap >> frame;
        if (frame.empty()) continue;

        // 在画面上绘制矩形和文字（模拟图像处理）
        cv::rectangle(frame, cv::Point(50, 50), cv::Point(100,
100),
```

```
                        cv::Scalar(0, 255, 0), 2);
        cv::putText(frame, "Loongson 2K300", cv::Point(10, 30),
                    cv::FONT_HERSHEY_SIMPLEX, 0.5, cv::Scalar(255,
0, 0), 1);

        // 发送处理后的图像
        udp.udp_send_image(frame, 50);
    }

    return 0;
}
```

六、波形数据传输示例

该示例实现编码器数据的实时采集与 UDP 传输，可配合上位机绘制波形。

```
c++
#include "lq_udp_client.hpp"
#include "lq_reg_pwm_encoder.hpp"
#include <stdio.h>
#include <unistd.h>

const std::string TARGET_IP   = "192.168.43.98";
const uint16_t    TARGET_PORT = 8080;

int main()
{
    printf("========================================\n");
    printf("  UDP Encoder Waveform\n");
    printf("========================================\n");

    // 初始化 UDP 客户端
    lq_udp_client udp;
    udp.udp_client_init(TARGET_IP, TARGET_PORT);
    printf("UDP client initialized\n");

    // 初始化编码器（根据实际引脚修改）
    ls_encoder_pwm enc1(ENC_PWM0_PIN64, PIN_72);
    ls_encoder_pwm enc2(ENC_PWM1_PIN65, PIN_73);
    printf("Encoder initialized\n");

    printf("Start streaming... Press Ctrl+C to stop\n");
```

```
    while (true) {
        // 读取两个通道的编码器值
        float ch1 = enc1.encoder_get_count();
        float ch2 = enc2.encoder_get_count();

        // 格式化编码器数据（适配上位机波形显示格式）
        char encoder_str[64];
        snprintf(encoder_str, sizeof(encoder_str),
                 "ch1:%7.2f,ch2:%7.2f", ch1, ch2);

        // 发送编码器数据
        udp.udp_send_string(encoder_str);

        // 打印当前数据
        printf("Encoder: %s\n", encoder_str);

        usleep(10000);  // 10ms 发送一次，保证实时性
    }

    return 0;
}
```

七、UDP 数据协议说明

7.1 字符串数据协议

采用简单的字符串格式，直接发送字符串内容，接收端可直接解析，适用于简单数据传输。

发送格式示例：

```plain text
"ch1:123.45,ch2:678.90"
"temp:25.5,hum:60.0,pres:101.3"
```

接收端解析方法：根据","分割字符串，再根据":"分割键值对，提取对应数据。

7.2 图像数据协议

图像数据采用自定义协议发送，确保接收端能正确解析，格式如下：

```plain text
```

```
┌──────────────┬──────────────────────┐
│  4 字节长度  │      JPEG 数据       │
└──────────────┴──────────────────────┘
```

字段说明：

- 长度字段：采用小端格式，标识后续 JPEG 数据的字节数
- JPEG 数据：通过 OpenCV imencode 函数编码后的图像数据

接收端处理步骤：

1. 先接收 4 字节长度数据，解析出 JPEG 数据的实际长度
2. 根据解析出的长度，接收后续的 JPEG 数据
3. 将 JPEG 数据解码为图像（如 OpenCV 的 imdecode 函数）

八、网络配置

8.1 查看本机 IP

在龙芯 2K300/301 开发板上执行以下命令，查看以太网或 WiFi 接口的 IP 地址：

```bash
ifconfig
```

或

```bash
ip addr
```

8.2 防火墙配置

若接收端为 PC，需开放对应 UDP 端口（如 8080），否则会导致数据接收失败。

【Windows 防火墙】

- 控制面板 → 系统安全 → 防火墙 → 高级设置
- 入站规则 → 新建规则 → 端口 → UDP → 特定端口（如 8080）
- 按照向导完成配置，允许该端口的入站连接

【Linux 防火墙】

```bash
sudo ufw allow 8080/udp
```

8.3 网络测试

【测试连通性】

在开发板上 ping 接收端 PC 的 IP，确认网络连通：

```bash
ping 192.168.43.98
```

【使用 netcat 测试 UDP 通信】

- PC 端（接收端）：打开终端，执行命令监听端口
- 开发板端（发送端）：执行命令发送测试数据

若 PC 端能接收到"test"字符串，说明 UDP 通信正常。

九、注意事项

- 1. 网络连接
  - 确保开发板和接收端在同一局域网内
  - 确认目标 IP 地址和端口号填写正确，避免因地址错误导致通信失败
  - 通信前需检查防火墙设置，确保端口已开放
- 2. 数据大小限制
  - UDP 协议单次发送数据长度有限制，通常不超过 65507 字节
  - 发送图像等大数据时，建议压缩后发送（如 JPEG 压缩）
  - 若需发送超过限制的大数据，需自行实现分包发送与拼接逻辑
- 3. 传输可靠性
  - UDP 协议不保证数据传输的可靠性，可能出现丢包、乱序、重复等情况
  - 适用于对实时性要求高、对可靠性要求不高的场景（如视频流、实时传感器数据）
  - 重要数据建议使用 TCP 协议，或在 UDP 基础上添加自定义确认、重传机制
- 4. 性能优化

- 降低图像分辨率和 JPEG 编码质量，可减少数据量，提升传输速度
- 根据实际需求调整发送间隔，避免频繁发送导致网络拥堵
- 避免在程序中频繁初始化 UDP 客户端，减少资源消耗

- 5. 线程安全
  - lq_udp_client 类支持线程安全，可在多线程环境中使用
  - 多线程同时调用发送/接收函数时，无需额外添加互斥锁
- 6. OpenCV 依赖
  - 使用 udp_send_image()函数发送图像时，需依赖 OpenCV 库
  - 编译代码时需定义 LQ_HAVE_OPENCV 宏，确保 OpenCV 相关代码正常编译

十、常见问题

Q1: 发送数据失败，返回负数?

A1: 检查以下几点:

- 开发板与接收端网络连接是否正常（可通过 ping 命令测试）
- 目标 IP 地址和端口号是否正确，是否与接收端监听端口一致
- UDP 套接字是否创建成功，初始化函数是否正常调用

Q2: 接收端收不到数据?

A2: 检查以下内容:

- 网络连通性：开发板 ping 接收端 IP，确认网络正常
- 防火墙设置：接收端是否开放了对应的 UDP 端口
- 监听端口：接收端是否在正确的端口上监听 UDP 数据
- 发送端配置：目标 IP 和端口是否填写错误

Q3: 图像传输卡顿、延迟严重?

A3: 尝试以下优化方法:

- 降低图像分辨率（如从 320×240 改为 160×120）
- 降低 JPEG 编码质量（如从 80 改为 30-50）
- 增加发送间隔，减少数据发送频率
- 检查网络环境，避免网络拥堵

Q4: 如何快速测试 UDP 通信是否正常？

A4: 使用 netcat 工具快速测试，步骤如下：

- PC 端（接收端）：执行命令 `nc -ul 8080`，监听 8080 端口
- 开发板端（发送端）：在代码中调用 `udp.udp_send_string("test");` 发送测试数据
- 若 PC 端能接收到"test"字符串，说明 UDP 通信正常

Q5: 接收函数 udp_recv()一直阻塞，无法退出？

A5: 可通过以下方式解决：

- 设置套接字超时：使用 setsockopt 函数设置 SO_RCVTIMEO 选项，指定超时时间
- 使用非阻塞模式：将套接字设置为非阻塞，避免接收时一直等待
- 使用 select/poll 多路复用：同时监听多个套接字，提升程序响应性

十一、配套上位机

本开源库的 UDP 传输功能可配合龙邱提供的上位机（LoongHost.exe）使用，实现数据可视化。

上位机核心功能

- 支持 UDP 图像显示（接收 udp_send_image()发送的图像数据）
- 支持多通道波形绘制（接收"ch1:值,ch2:值"格式的字符串数据）
- 支持多通道数据实时显示，便于调试

数据格式要求：

- 图像数据：必须使用 udp_send_image() 函数发送，遵循自定义图像协议
- 波形数据：必须按照 "ch1:值,ch2:值" 的格式发送字符串

十二、参考资源

- UDP 客户端头文件：
  - libraries/drv/inc/lq_udp_client.hpp
- UDP 客户端实现源码：
  - libraries/drv/src/lq_udp_client.cpp

- 示例代码:
  - example/src/12_lq_udp_img_trans_demo.cpp（图像传输示例）
  - example/src/13_lq_udp_wavefrom_demo.cpp（波形数据传输示例）